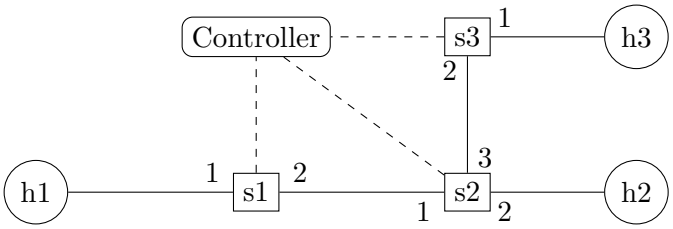
Piattaforme Software per la Rete (Modulo 1) – 01/07/2014

Su ogni foglio consegnato scrivere nome, cognome, matricola e il numero dell'esercizio.

Esercizio 1

Si consideri la rete in figura. I nodi h1,. . . ,h3 contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,. . . ,h3 sono MAC1,. . . ,MAC3 e IP1,. . . ,IP3. Sui nodi h1, h2 e h3 sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse.



Il controllore POX è collegato ai nodi s1,. . . ,s3 ed è configurato come indicato nel Programma 1.

Scrivere tutti i pacchetti che sono inviati su tutti i collegamenti della rete e tra i nodi e il controllore in coseguenza degli eventi seguenti. Scrivere il contenuto delle tabelle openflow prima e dopo ogni evento.

(a) Apertura di una connessione TCP da h2 a h3 dalla porta 12345 alla porta 80;

Solution:

All'accensione gli switch sono configurati come segue:

Switch 1

match	action	statistics
dl_dst = MAC2	output interface 2	0

Switch 2

match	action	statistics
dl_dst = MAC2	output interface 2	0

Switch 3

match	action	statistics
dl_dst = MAC2	output interface 2	0
dl_src = MAC3 tp_dst = 80	tp_dst rewrite 437 output interface 2	0
dl_dst = MAC3 tp_src = 80	tp_dst rewrite 80 output interface 1	0

Programma 1

```
from pox.core import core
import pox.openflow.libopenflow_01 as of

def _handle_ConnectionUp(event):
  msg = of.ofp_flow_mod()
  msg.dl_dst = MAC2
  msg.actions.append(of.ofp_action_output(2))
  event.connection.send(msg)
  if (event.dpid == 3): #event.dpid contiene il numero dello switch
    msg = of.ofp_flow_mod()
    msg.dl_src = MAC3
    msg.tp_dst = 80
    msg.actions.append(of.ofp_action_tp_port.set_dst(437))
    msg.actions.append(of.ofp_action_output(2))
    event.connection.send(msg)
    msg = of.ofp_flow_mod()
    msg.dl_dst = MAC3
    msg.tp_src = 437
    msg.actions.append(of.ofp_action_tp_port.set_src(80))
    msg.actions.append(of.ofp_action_output(1))
    event.connection.send(msg)

def _handle_PacketIn(event):
  packet = event.parsed
  if (event.dpid == 1) and (packet.dst == MAC2):
    udp_p = packet.find('udp') # estrae il datagramma udp
    if udp_p: # e' un pacchetto udp
      if udp_p.dstport=53:
        msg = of.ofp_flow_mod()
        msg.nw_dst = IP2
        msg.nw_proto = 17 #UDP
        msg.tp_src = udp_p.srcport
        msg.tp_dst = 53
        msg.actions.append(of.ofp_action_dl_addr.set_dst(MAC3))
        msg.actions.append(of.ofp_action_nw_addr.set_dst(IP3))
        msg.actions.append(of.ofp_action_output(2))
        msg.buffer_id = event.ofp.buffer_id # applica subito
        event.connection.send(msg)
        return
  msg = of.ofp_packet_out(data = event.ofp)
  msg.actions.append(of.ofp_action_output(of.OFPP_FLOOD))
  event.connection.send(msg)

def launch():
  core.openflow.addListenerByName("ConnectionUp", _handle_ConnectionUp)
  core.openflow.addListenerByName("PacketIn", _handle_PacketIn)
```

tcp syn	
h2 → s2	MAC2:MAC3:IP2:IP3:TCP:12345:80
s2 → c	packet_in(data=MAC2:MAC3:IP2:IP3:TCP:12345:80)
c → s2	packet_out(action=OFPP_FLOOD)
s2 → s3,s1	MAC2:MAC3:IP2:IP3:TCP:12345:80
s3 → c	packet_in(data=MAC2:MAC3:IP2:IP3:TCP:12345:80)
c → s3	packet_out(action=OFPP_FLOOD)
s3 → h3	MAC2:MAC3:IP2:IP3:TCP:12345:80
s1 → c	packet_in(data=MAC2:MAC3:IP2:IP3:TCP:12345:80)
c → s1	packet_out(action=OFPP_FLOOD)
s1 → h1	MAC2:MAC3:IP2:IP3:TCP:12345:80

tcp syn ack	
h3 → s3	MAC3:MAC2:IP3:IP2:TCP:80:12345
s3 → s2	MAC3:MAC2:IP3:IP2:TCP:80:12345
s2 → h2	MAC3:MAC2:IP3:IP2:TCP:80:12345

tcp ack	
h2 → s2	MAC2:MAC3:IP2:IP3:TCP:12345:80
s2 → c	packet_in(data=MAC2:MAC3:IP2:IP3:TCP:12345:80)
c → s2	packet_out(action=OFPP_FLOOD)
s2 → s3	MAC2:MAC3:IP2:IP3:TCP:12345:80
s3 → c	packet_in(data=MAC2:MAC3:IP2:IP3:TCP:12345:80)
c → s3	packet_out(action=OFPP_FLOOD)
s3 → h3	MAC2:MAC3:IP2:IP3:TCP:12345:80
s1 → c	packet_in(data=MAC2:MAC3:IP2:IP3:TCP:12345:80)
c → s1	packet_out(action=OFPP_FLOOD)
s1 → h1	MAC2:MAC3:IP2:IP3:TCP:12345:80

Stato finale degli switch

Switch 1		
match	**action**	**statistics**
dl_dst = MAC2	output interface 2	0

Switch 2		
match	**action**	**statistics**
dl_dst = MAC2	output interface 2	1

Switch 3		
match	**action**	**statistics**
dl_dst = MAC2	output interface 2	1
dl_src = MAC3 tp_dst = 80	tp_dst rewrite 437 output interface 2	0
dl_dst = MAC3 tp_src = 80	tp_dst rewrite 80 output interface 1	0

(b) Invio di un datagramma UDP da h1 a h3 dalla porta 5432 alla porta 53 e relativa risposta;

Solution:
Stato iniziale da soluzione precedende

domanda	
h1 → s1	MAC1:MAC3:IP1:IP3:UDP:5432:53
s1 → c	packet_in(data=MAC1:MAC3:IP1:IP3:UDP:5432:53)
c → s1	packet_out(action=OFPP_FLOOD)
s1 → s2	MAC1:MAC3:IP1:IP3:UDP:5432:53
s2 → c	packet_in(data=MAC1:MAC3:IP1:IP3:UDP:5432:53)
c → s2	packet_out(action=OFPP_FLOOD)
s2 → s3,h2	packet_in(data=MAC1:MAC3:IP1:IP3:UDP:5432:53)
s3 → c	packet_in(data=MAC1:MAC3:IP1:IP3:UDP:5432:53)
c → s3	packet_out(action=OFPP_FLOOD)
s3 → h2	MAC1:MAC3:IP1:IP3:UDP:5432:53

risposta	
h3 → s3	MAC3:MAC1:IP3:IP1:UDP:53:5432
s3 → c	packet_in(data=MAC3:MAC1:IP3:IP1:UDP:53:5432)
c → s3	packet_out(action=OFPP_FLOOD)
s3 → s2	MAC3:MAC1:IP3:IP1:UDP:53:5432
s2 → c	packet_in(data=MAC3:MAC1:IP3:IP1:UDP:53:5432)
c → s2	packet_out(action=OFPP_FLOOD)
s2 → s1,h2	packet_in(data=MAC1:MAC3:IP1:IP3:UDP:5432:53)
s1 → c	packet_in(data=MAC3:MAC1:IP3:IP1:UDP:53:5432)
c → s1	packet_out(action=OFPP_FLOOD)
s1 → h1	MAC3:MAC1:IP3:IP1:UDP:53:5432

Stato finale degli switch immutato

(c) Invio di un nuovo datagramma UDP da h1 a h2 dalla porta 5432 alla porta 53 e relativa risposta.

Solution:
Stato iniziale da soluzione precedende

domanda	
h1 → s1	MAC1:MAC2:IP1:IP2:UDP:5432:53
s1 → s2	MAC1:MAC2:IP1:IP2:UDP:5432:53
s2 → h2	MAC1:MAC2:IP1:IP2:UDP:5432:53

risposta	
h2 → s2	MAC2:MAC1:IP2:IP1:UDP:53:5432
s2 → c	packet_in(data=MAC2:MAC1:IP2:IP1:UDP:53:5432)
c → s2	packet_out(action=OFPP_FLOOD)
s2 → s1,h2	packet_in(data=MAC2:MAC1:IP2:IP1:UDP:53:5432)
s1 → c	packet_in(data=MAC2:MAC1:IP2:IP1:UDP:53:5432)
c → s1	packet_out(action=OFPP_FLOOD)
s1 → h1	MAC2:MAC1:IP2:IP1:UDP:53:5432

Stato finale degli switch

Switch 1

match	**action**	**statistics**
dl_dst = MAC2	output interface 2	1

Switch 2

match	**action**	**statistics**
dl_dst = MAC2	output interface 2	2

Switch 3

match	**action**	**statistics**
dl_dst = MAC2	output interface 2	1
dl_src = MAC3 tp_dst = 80	tp_dst rewrite 437 output interface 2	0
dl_dst = MAC3 tp_src = 80	tp_dst rewrite 80 output interface 1	0

Esercizio 2

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
A	*	80	*	TCP
B	D	21	22	TCP
*	E	*	53	UDP
B	*	*	123	*
C	F	23	*	UDP
A	D	25	80	*

(a) Definire il bitvector set per ciascun campo.

Solution:

Source Address		Dest. Address		Source Port		Dest. Port		Protocol	
A	101001	D	110101	21	011100	22	110010	TCP	110101
B	011100	E	101100	23	001110	53	101010	UDP	001111
C	001010	F	100110	25	001101	80	100011	*	000101
*	001000	*	100100	80	101100	123	100110		
				*	001100	*	100010		

(b) Assumendo che gli indirizzi sorgente/destinazione siano codificati su 32 bit, i numeri di porta sorgente/destinazione su 16 bit e il tipo di protocollo su 8 bit, calcolare la dimensione totale (in bit) del bitvector set.

Solution: (32+6)*8+(16+6)*10+(8+6)*3= 566 bit

(c) Si considerino i pacchetti (H, E, 32, 80, ICMP) e (B, G, 21, 123, TCP). Applicare ad entrambi la procedura di BVLS per identificare gli eventuali matching.

Solution: 1) 001000 AND 101100 AND 001100 AND 100010 AND 000101 = 000000 Nessuna regola soddisfatta. 2) 011100 AND 100100 AND 011100 AND 100110 AND 000101 = 000100 Il pacchetto soddisfa la regola 4.

(d) Si assuma l'utilizzo di summary per parole di 3 bit. Si applichi nuovamente la procedura di BVLS al pacchetto 1 utilizzando i summary. Si confronti il numero di bit letti da memoria in questo caso, rispetto al numero di bit letti al punto 2. L'utilizzo di summary è risultato vantaggioso?

Solution: Al punto 2 la procedura di BVLS richiede la lettura di 5 stringhe da 6 bit ciascuna, per un totale di 30 bit. Nel caso vengano utilizzati summary su parole da 3 bit, vengono inizialmente lette 5 stringhe da 2 bit ciascuna, per un totale di 10 bit. Nel caso specifico, il risultato dell'AND sui 5 summary è: 10 AND 11 AND 11 AND 11 AND 01=00. Poichè il risultato è una stringa di zeri, nessuna regola può essere applicata al pacchetto, e non c'è necessità di accedere alle parole intere. Quindi sono stati letti 20 bit in meno.

Esercizio 3

Una matrice di commutazione crossbar di uno switch 3×3 usa un algoritmo per lo scheduling delle trasmissioni. Si assuma pacchetti tutti di lunghezza L,

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

Ingresso 1: 2, 3, 1

Ingresso 2: 1, 2, 3

Ingresso 3: 1, 3, 1

(a) Svolgere l'algoritmo Take-a-Ticket (TaT) fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali ticket sono assegnati. L'algoritmo di assegnazione dei ticket privilegia sempre l'interfaccia di ingresso con l'identificativo più piccolo.

Solution:

Round		Ingresso 1	2	3
1	Assegnazione	(1,1)	(3,1)	(1,2)
	Trasmissione	1	3	–
2	Assegnazione	(3,2)	(2,1)	–
	Trasmissione	3	2	1
3	Assegnazione	(2,2)	(1,3)	(3,3)
	Trasmissione	2	1	3
4	Assegnazione	–	–	(1,4)
	Trasmissione	–	–	1

(b) Svolgere l'algoritmo PIM fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali grant sono richiesti e sono assegnati. L'algoritmo privilegia sempre l'interfaccia di ingresso o di uscita con l'identificativo più piccolo.

Solution:

Round		Ingresso 1	2	3
1	Richiesta	{1,2,3}	{1,2,3}	{1,3}
	Assegnazione	{1,2,3}	–	–
	Trasmissione	1	–	–
2	Richiesta	{2,3}	{1,2,3}	{1,3}
	Assegnazione	{2,3}	{1}	–
	Trasmissione	2	1	–
3	Richiesta	{3}	{2,3}	{1,3}
	Assegnazione	{3}	{2}	{1}
	Trasmissione	3	2	1
4	Richiesta	–	{3}	{1,3}
	Assegnazione	–	{3}	{1}
	Trasmissione	–	3	1
5	Richiesta	–	–	{3}
	Assegnazione	–	–	{3}
	Trasmissione	–	–	–

(c) Facendo riferimento all'algoritmo TaT con assegnazione casuale, si consideri uno scenario di traffico uniforme e code sempre sature. In quale caso la matrice lavora alla massima efficienza, ovvero tutte le uscite sono usate? Qual è la probabilità che ciò accada?

Solution: Quando i tre ingressi hanno pacchetti per tre uscite distinte.

$$p = \frac{3!}{3^3} = \frac{6}{27} \simeq 22\%$$

(d) Si consideri la variante dell'algoritmo TaT in cui i pacchetti che trovano l'interfaccia d'uscita occupata sono inviati alla prima interfaccia libera disponibile (*deflection routing*), anche se non corretta. Stimare la percentuale di pacchetti deflessi.

Solution:

0 pacchetti deflessi tre pacchetti per tre uscite distinte

$$\Pr[0] = \frac{3!}{3^3}$$

2 pacchetti deflessi tre pacchetti per la stessa uscita

$$\Pr[2] = \frac{3}{3^3}$$

1 pacchetto deflesso due pacchetti per la stessa uscita e uno per un'uscita diversa

$$\Pr[1] = \frac{3^3 - 3! - 3}{3^3}$$

$$p = \frac{3^3 - 3! - 3}{3^3} + 2\frac{3}{3^3} = 1 - \frac{1}{9} = 0.89$$

(e) Facendo riferimento all'algoritmo PIM con scelta casuale, si consideri uno scenario di traffico uniforme e code infinite. In quale caso la matrice lavora alla massima efficienza, ovvero tutte le uscite sono usate? Qual è la probabilità che ciò accada?

Solution: Quando le tre uscite assegnano i grant a tre ingressi distinti.

$$p = \frac{3!}{3^3} = \frac{6}{27} \simeq 22\%$$